Authority-Aligned Linearization for Rescue-Service Incident Response over Intermittent Edge Networks

Anonymous Author(s)
Anonymous Affiliation
Submitted to NCA 2026 (double-blind review)

***Abstract*—In rescue-service incident response, one principal holds operational command authority at every moment, yet responders must keep working through responder-side and backhaul partitions. Authority-bearing decisions must therefore stay attributable to that single commander even under intermittent connectivity. We present authority-aligned linearization (AAL), a consistency discipline. AAL places the incident-journal linearization point at the edge node that currently holds command authority, so authority-bearing writes serialize at that node. Non-authority data (reference state, field submissions, materialized views) stays locally available and is forwarded later, and command transfer uses fenced promotion rather than leader election. We implement AAL as a Docker Compose prototype, model its single-authority and fenced-promotion invariants in TLA+ and Hypothesis, and evaluate it under emulated partitions against a Hanssen-style CRDT-LWW baseline and a Raft consensus baseline. The comparison shows why standard distributed-systems tools do not fit this setting. Conflict-free replicated data type (CRDT) merge is mergeable-by-default, so it cannot encode authority for non-commutative decisions. Raft correctly moves the writer to whatever quorum is reachable. Neither lets an operator designate the commander. Under AAL, partitions on either responder or backhaul links do not block command-edge progress, and stale-epoch traffic is rejected before the journal write, costing 2.5× less than a successful commit. The cost is no automatic failover: promotion is operator-initiated.**

***Index Terms*—authority-aligned linearization, network architectures and protocols, edge computing, intermittent connectivity, distributed systems, incident response, prototype evaluation**

I. Introduction

Public-safety information systems must remain operational when commercial connectivity fails [1]–[3]. Rescue-service vehicles operate up to several hours away from regional dispatch centres. In that terrain, wireless connectivity alternates between full wide-area coverage, mesh-only operation, and total isolation [4]–[6]. Disasters routinely degrade or destroy commercial telecom infrastructure precisely when coordination demand is highest [7], as documented in the 2018 Swedish forest-fire crisis [8]. Vehicles must continue acting on stale or partial information without external coordination. Field submissions entered on tablets must survive hours without connectivity. In command-hierarchical environments the consequences of information loss are wrong assignments, missed hazards, and conflicting orders [9].

The operational constraints in this paper are grounded in public Swedish rescue-service guidance: Räddningstjänsten Storgöteborg's incident-planning and municipal rescue-plan guidance [10], [11]. That guidance treats incident-planning information as an ongoing, partly secrecy-restricted process handled in operational mobile IT support rather than a static paper artifact. Independent academic work on Swedish rescue-service information-systems practice corroborates these end-user requirements and information-flow shortcomings [12], [13]. From these public sources we derive three requirements and two deployment constraints. The requirements are R1 offline operation, R2 durable forward-only field submission, and R3 attributable command authority. The deployment constraints are avoiding cross-vehicle broadcast hazards and reusing existing rescue-service certificate and identity infrastructure. § II-A separates the externally motivated requirements from design assumptions and integration choices.

The design question is: *how should authority-bearing data be synchronized across intermittently connected edge tiers, when the operational model designates a single principal as authoritative?* The answer is authority-aligned linearization (AAL), a method that places the incident-journal linearization point at the command edge, the vehicle edge node that currently holds operational command authority. We evaluate the resulting AAL design with a prototype that combines core-to-edge reference-data distribution, responder outboxes (durable local queues that hold field submissions until command accepts them), a single incident journal, and fenced promotion. The contribution is not a new consensus or CRDT mechanism. It is a problem analysis that justifies a design: a characterization of where the linearization boundary should sit for this class of rescue-service workflow, and why the standard distributed-systems toolkit does not match this workflow well. We make three contributions:

1) (C1) a problem analysis of authority-bearing incident coordination under intermittent connectivity. It positions local-first systems, CRDT-based replication, and consensus-based state-machine replication against the rescue-service operational requirements (R1–R3). It shows where each one is insufficient or over-engineered (does more than the workload needs): the workload is mostly lookups plus append-only incident reporting, yet it still requires attributable command authority;
2) (C2) an authority-aligned synchronization model based on a single authoritative incident journal and fenced promotion, with a TLA+ specification, corresponding Hypothesis state-machine tests, and a service-path mechanism that rejects stale-epoch traffic at the command edge;

3) (C3) a prototype implementation (Docker Compose, *tc-netem*) and a partition-based evaluation that compares AAL against Hanssen-style CRDT-LWW and Raft consensus baselines—showing authority remains with the operator-designated edge independent of reachability, where CRDT cannot encode it and Raft follows the reachable quorum—alongside measured propagation, wide-area recovery, sequencing, duplicate replay, stale-epoch rejection, and strict-versus-weak promotion checks that falsify unfenced variants.

II. Authority-Bearing Incident Coordination under Intermittent Connectivity

Rescue-service incident response exposes a design tension between availability and authority. Responders must continue operating under intermittent connectivity, yet authority-bearing incident decisions must remain attributable to a single operational commander.

The target setting is municipal rescue-service incident response, where a regional core supports dispatch and reference data, vehicle edge nodes operate near the incident site, and field tablets attach to local vehicles acting as leaf clients. Connectivity is intermittent: vehicles may move between wide-area connectivity, mesh-only contact, and total isolation [4], [5]. One vehicle edge node holds the command role for the incident; we refer to it as the *command edge*. The role can move to another vehicle, but only through an explicit human decision, and the protocol must respect that operational authority.

Figure 1 illustrates the actors and the three connectivity regimes.

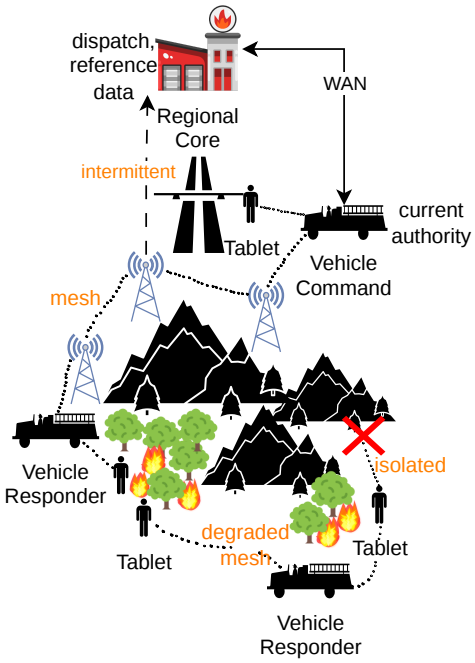

Fig. 1. Operational setting. A regional core supports dispatch and reference data; vehicle edge nodes run near the incident site; tablets attach to a local vehicle. One vehicle holds command authority at any instant. Vehicles simultaneously occupy different connectivity regimes: wide-area backhaul, intermittent backhaul with mesh transit, or total isolation.

A. Operational Requirements

Public rescue-service guidance and studies of Swedish rescue-service information systems motivate the requirements used here [10]–[13]. These requirements are externally motivated and are stated independently of any particular architecture; § II-C separates them from the design choices made in response. The positioning in § II-B is argued relative to them.

The externally motivated requirements are:

- ***R1 (Offline operation)*** Each tier must keep working from previously synchronized data without continuous connectivity.
- ***R2 (Durable forward-only submission)*** Field submissions entered on tablets must survive vehicle restart and prolonged partition.
- ***R3 (Attributable command authority)*** Resource assignments, status transitions, and hazard annotations must be attributable to one operational command authority at any instant.

R3 should not be read as a claim that crisis governance is simple. Incident-command research describes a command hierarchy operating inside multi-actor response networks, where authority can be shared, contested, delegated, or bounded by legal and organizational relationships [14]–[17]. For AAL, this motivates conservative transfer semantics: the protocol accepts a new command edge only after an operator-initiated promotion record, rather than inferring authority from reachability.

B. Positioning Relative to Consensus, CRDTs, Local-first, and Emergency Networking

AAL recombines well-known mechanisms (single-leader replication, transactional outbox [18], idempotency keys, manual fencing) under a domain-specific rule: the incident-journal linearization point is the command edge, set by an external operational protocol rather than chosen by liveness-driven leader election. The incident-state entity owns its decision history; external actors interact through forward-only events rather than mutate state directly [19]. The novelty is therefore in (i) which operations are placed under that linearization point and which are not, and (ii) the fenced-promotion rule that preserves single-authority across command changes; the mechanisms themselves are not claimed as novel. This subsection argues that each standard distributed-systems family either fails or over-serves R1–R3, motivating a linearization boundary aligned with command authority.

Consensus and replicated state machines. Consensus, shared-log, and chain-replication systems provide replicated agreement and totally ordered logs under their stated failure models [20]–[23]. These systems supply a stronger ordering property than R3 requires, at a cost the workload does not justify: rescue-service traffic is dominated by reference-data lookups and append-only incident reporting, for which quorum agreement and leader election are over-engineered. Election-

based liveness selects a writer by reachability, whereas R3 fixes the writer by operational command; § IV-E1 measures this mismatch on a three-voter Raft baseline. A partition that correctly moves Raft leadership to a reachable majority is not the same as an operator-designated command transfer.

CRDTs and strong eventual consistency. CRDT and strong-eventual-consistency work, including richer local-first safety models and Hanssen's CRDT replication for emergency management, our closest baseline, provides convergence for mergeable replicated state [24]–[30]. This serves R1 (offline operation) and R2 (durable local submission) well, but cannot satisfy R3 [31]. Authority-bearing incident decisions—resource assignments, status transitions, hazard annotations—are non-commutative. There is no application-defensible merge for two responders setting the same status to conflicting values. Any deterministic resolution rule implicitly invents the authority that R3 requires be explicit; AAL instead makes that authority operational rather than emergent.

Offline-first and local-first systems. Offline-first and local-first systems prioritize local availability and continued operation under poor connectivity [18], [32], [33]. They satisfy R1 and R2 and are the right model for reference data, field submissions, and materialized views. But their design goal is replica *autonomy*—every node is equally entitled to write and reconcile—which is precisely the property the command hierarchy rejects for authority-bearing decisions (R3).

Emergency and edge networking. Closest in setting is emergency and edge networking. CoNICE [34] is a concrete consensus design for intermittently connected emergency response: it reaches agreement among infrastructure-less peers using One-Third-Rule consensus, and its agreement level is the strongest/highest-complexity option, layered over epidemic replication, causality metadata, and decision-invalidation support for long-term fragmentation. That is substantial machinery when the operational problem already designates a commander. Rescue-service incident command, by contrast, has a designated commander at every moment, and the network is not permitted to vote on incident decisions, so CoNICE's no-privileged-peer assumption is exactly the one R3 removes. Public-safety networking and edge/fog research place resilient transport and storage near responders [5], [35], [36], addressing R1 transport but not which actor may issue an authority-bearing decision.

Across these families, each helps responders keep working or order writes, but none decides *which actor* may issue an authority-bearing decision during a partition. We are not aware of prior work that explicitly uses operational command authority as the linearization boundary while assigning weaker contracts to non-authority incident data.

C. Assumptions and Design Choices

The requirements in § II-A are externally motivated; the following choices are this paper's response to them and could in principle be made differently. Most importantly, attributable command authority (R3) does not logically force a single-writer database. R3 is a requirement on attribution; aligning the incident-journal linearization boundary with command authority is the design choice this paper adopts to meet it. The design assumes that an incident has one current command authority; the command role is fixed at dispatch and changes only through an explicit operator decision.

The deployment constraints identified in the introduction are separate: the mesh should act as routed transit rather than a stretched user network, and the system must reuse the rescue-service identity infrastructure. Their architectural consequences—edge cache and local services, tablet/edge outbox and retry, a single incident-journal writer, mesh-as-transit routing, and mTLS/CA identity integration—are developed in § III.

III. Authority-Aligned Architecture and Protocol

The requirements imply that not all incident data should be treated uniformly: offline operation and durable field submission favor local availability and eventual synchronization, while attributable command authority requires a single authoritative ordering of authority-bearing decisions.

A. State Classes and Consistency Contracts

A central architectural decision in AAL is that different classes of incident data require different consistency guarantees. This split is a RedBlue-style [37] consistency partition. Authority-bearing decisions go in the strong set; forward-only field submissions, cached reference data, and materialized state go in the weaker classes. Table I summarizes the main state classes.

Master and reference data are owned by the regional core and cached at edges. Their partition behavior is stale-but-available local read. Field submissions are durable at the responder first. They become authoritative only after command acceptance. Authority-bearing incident events are sequenced by the command edge with a command-assigned *event_seq*. Materialized state is derived from the append-only incident journal.

Artifacts and attachments use immutable or manifest-based distribution. Audit records and command-to-core backhaul are forwarded eventually. Promotion and fencing metadata are part of the command-authority protocol. The current evidence for promotion/fencing is model-level; § III-E gives the protocol rule.

B. Tiered Edge Topology

The regional core provides master/reference data and receives backhaul. Vehicle edges provide local services during disconnection. Responder vehicles queue submissions durably. The command edge owns the append-only incident journal. Tablets are leaf clients and do not participate directly in command sequencing.

The mesh is transit, not the consistency mechanism. User VLANs are not extended across vehicles. Application services communicate between edge nodes over routed links in the target deployment. The state classes of § III-A already fix the

TABLE I
ONLY AUTHORITY-BEARING JOURNAL ENTRIES USE THE SINGLE-WRITER CONTRACT; OTHER INCIDENT STATE USES LOCAL OR EVENTUAL SYNCHRONIZATION CONTRACTS.

State	Authority/source	Synchronization contract
Reference data	regional core	One-way core-to-edge replication; eventual convergence.
Immutable artifacts	publisher / core	Manifest, package, or hash validation where implemented.
Field observations	tablet / responder	Locally durable outbox; at-least-once forwarding; idempotent accept.
Command decisions	command edge	Single-writer rule applies only here; total order by *event_seq*.
Materialized state	journal	Derived and rebuildable; not authoritative.
Audit events	edge / core	Eventual forwarding; validation incomplete.

linearization boundary (M1); the protocol adds three mechanisms over it—incident-journal sequencing (M2), durable outbox forwarding (M3), and fenced promotion (M4).

The main identifiers are *incident_id*, a client-supplied *client_event_id*, and a command-assigned *event_seq*. The promotion model also uses a command epoch (a numbered command tenure, where a higher number means newer authority).

C. Incident Journal Sequencing (M2)

M2 specifies how the command edge linearizes authority-bearing operations into a totally ordered journal. The command edge accepts a *client_event_id* once and assigns *event_seq* atomically with journal insertion, mapping duplicates to the existing journal entry rather than resequencing them. Consumers apply committed entries in *event_seq* order; materialized state is derived from the journal and can be rebuilt.

D. Durable Outbox and Idempotent Submission (M3)

M3 specifies the responder-side path for non-authority data: locally durable, idempotent, and eventually forwarded to the incident journal. Figure 2 shows the steady-state field-submission path. The boundary is between local durability and authoritative command commit. A responder may acknowledge local acceptance after the outbox insert. It marks an item forwarded only after incident-journal acceptance or duplicate recognition.

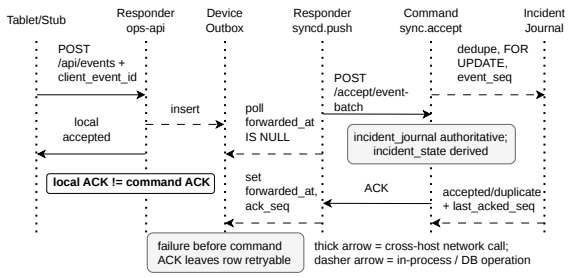

Fig. 2. Responder outbox protocol: the steady-state field-submission path from tablet to incident journal. The journal assigns the authoritative sequence number, and duplicate delivery is handled by *client_event_id*.

The M3 contract has three parts:

1) Each submission has a globally unique idempotency key, *client_event_id*, scoped by the incident and originating client identity.
2) A responder accepts a submission only after the event is stored in its local outbox.
3) A responder may mark an outbox item forwarded only after the command edge acknowledges durable incident-journal acceptance or duplicate recognition.

Outbox delivery is at least once and is deduplicated at the command edge; there is no claim of causal ordering across independent responder outboxes before command assignment.

E. Fenced Promotion (M4)

M4 specifies fenced promotion, the mechanism used to transfer command authority without forking the journal. The design does not provide autonomous failover. An operator confirms fenced promotion, and fencing prevents stale command authority from continuing. Figure 3 summarizes the intended rule. Each command tenure has a monotonically increasing epoch or equivalent term. Promotion emits an explicit record. Stale command traffic must be rejected once a newer epoch is known. This strict rule sacrifices availability when the previous command cannot be fenced or its status cannot be established. In the prototype this rule preserves *SingleAuthority* and *NoForkedJournal*, and rejecting a stale epoch (median 2.5 ms) is cheaper than a commit; § IV-D reports the full service-path measurement and the strict-versus-weak comparison.

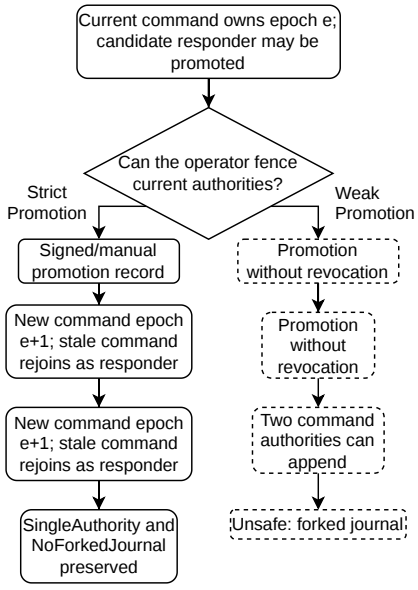

Fig. 3. Fenced promotion preserves the single-writer invariant. Left (single-border): with fencing, the stale command is forced to rejoin as a responder. Right (double-border): without fencing, two command authorities can append concurrently, producing a forked journal. The unsafe branch appears as a 3-state TLA+ trace for split authority and a 5-state trace for a forked journal; durable epoch storage and stale-epoch rejection are implemented in the prototype.

F. Partition Behavior and Promotion Blocking

If a responder is cut off from command, it can keep using cached data and store new submissions in its local outbox, but those submissions are not authoritative until the command edge accepts them and assigns *event_seq*. If command is cut off from the regional core, command-edge incident work continues and the core catches up later from the committed incident journal. If command must move to another vehicle, AAL does not elect a replacement automatically: an operator must promote the new command edge and prevent stale command authority from continuing to write, otherwise promotion is blocked.

IV. Prototype and Evaluation

A. Prototype and Emulation Setup

The artifact repository contains a prototype service path for the core and edge tiers, plus deployment scaffolding for the target vehicle environment. Python services target Python 3.12 and use FastAPI and asyncpg. Database schemas are managed through numbered SQL migrations. Docker Compose files instantiate the emulated core, command edge, and responder edges used in the emulation setup below; a separate `baseline/crdt` Compose stack provides the CRDT-LWW comparison point.

Table II records the prototype evidence boundary for each protocol concept from § III. We use three evidence levels throughout: *measured* (exercised on the prototype service path), *model-level* (established by the TLA+/Hypothesis checks but not the service path), and *deployment-only* (target-deployment design, not exercised here). Later references to the evidence boundary reuse this taxonomy.

TABLE II
Prototype evidence boundary for each protocol concept.

Concept	Status
M1–M2: Linearization boundary and incident-journal sequencing	
Incident journal and *event_seq*	Implemented; evaluated on the command-edge *syncd* accept path.
Command-edge accept during responder partition	Evaluated for the direct command-edge *syncd* accept path during responder isolation; tablet-facing command-edge *ops-api* not exercised.
M3: Durable outbox and idempotency	
Responder durable outbox	Implemented; evaluated for responder forwarding.
client_event_id idempotency	Implemented; evaluated by duplicate replay (§ IV-C).
M4: Fenced promotion	
Promotion and fencing	Durable command-epoch table and stale-epoch rejection implemented at service level; matching TLA+/Hypothesis model checks; measured in § IV-D.
Supporting infrastructure (not authority-aligned)	
Core bootstrap and backhaul	Synthetic bootstrap and incident-journal backfill; evaluated in Docker.
Vehicle edge roles	Command and responder edges in Compose.
Incident-journal forwarding to core	Implemented; evaluated for WAN recovery/backfill.
Tablet client	Python stub in evaluation; Android scaffold only.
Multi-condition netem evaluation	Implemented; measured under two link profiles in § IV-C.
Transit and security controls	Target-deployment documentation and configuration; physical mesh and security overhead are out of scope for the reported evaluation.

Security controls (VPN overlay, mTLS termination, certificate-derived identity) are deployment design, not measured here; the emulation uses plain HTTP on a Docker bridge with synthetic device and user identifiers. We evaluate the prototype on a single-host emulation, organized by the four mechanisms of § III: M1–M2 (the linearization boundary and incident-journal sequencing), M3 (the durable responder outbox), and M4 (fenced promotion). Each subsection names the requirement it serves, the claim it tests, and the evidence level—*measured*, *model-level*, or *deployment-only* (§ IV). Table III maps mechanisms to requirements at a glance. Three further experiments delimit what AAL does not promise rather than evaluate a mechanism: a Raft consensus baseline (§ IV-E1), a CRDT-LWW semantic falsification (§ IV-E2), and command-to-core WAN backhaul recovery (§ IV-E3).

Measurements come from a Docker Compose emulation with one regional core, one command edge, and three responder edges on a shared bridge. The propagation, recovery, and throughput cells use a single responder path; the other responders only populate the topology, so reported latencies are per-path, not aggregate. Each cell repeats $n = 30$ unless noted, with median and p95 reported. Partitions are injected with *tc* in a narrowly targeted way: only the targeted path is cut, so unrelated paths stay reachable and each cell isolates one effect. Impairment uses two *tc-netem* profiles informed by published mesh measurements [4], [35]: stressed (80 ms $\pm$ 30 ms delay, 2 % loss) and degraded (200 ms $\pm$ 80 ms delay, 8 % loss). Table IV collects the primary service-path numbers.

B. M1–M2: linearization boundary and journal sequencing

M1–M2 (the linearization boundary and incident-journal sequencing) serve R3, and R1 for command-side writes. Claim: authority-bearing writes commit independently of responder reachability and receive consecutive, non-duplicate *event_seq* values. Evidence: measured.

With one responder isolated for ten seconds, direct single-event POSTs to */accept/event-batch* all commit, assigning consecutive non-duplicate *event_seq* values at single-digit millisecond latency (Table IV). The endpoint requires a current *X-Command-Epoch*; stale or absent epochs are rejected before journal work. Responder reachability is therefore not in the commit path.

Saturation throughput at the linearization point (Table IV) reflects a single-writer serializer.

C. M3: durable outbox and idempotent submission

M3 (the durable responder outbox with *client_event_id* idempotency) serves R1 and R2. Claim: responder submissions

TABLE III

MECHANISM-TO-REQUIREMENT TRACEABILITY. EACH MECHANISM IS TRACED TO THE REQUIREMENT(S) IT SERVES, THE CLAIM TESTED, THE EXPERIMENT, THE KEY RESULT, AND ITS EVIDENCE LEVEL. ALL NUMBERS ARE REPRODUCED FROM TABLE IV, TABLE VI, AND FIG. 4.

R	M	Claim tested	Experiment	Key result	Evidence
R3, R1	M1–M2	Authority-bearing writes commit without responder reachability and stay totally ordered	Command accept under responder isolation; saturation throughput	20/20 commits, *event_seq* 1–20 gap-free, 0 duplicate seqs	measured
R1, R2	M3	Durable idempotent submissions reach the journal exactly once under retry, restart, and impairment	Propagation latency, duplicate replay, crash-restart, netem	median $\leq$931.5 ms; exactly 50 rows on replay; 50/50 survive restart ($\sim$2.6 s)	measured
R3	M4	Fenced promotion preserves single authority across transfer and rejects stale epochs cheaply	TLA+/Hypothesis bounds; service-path promotion	commit median 6.3 ms/5.7 ms, stale reject 2.5 ms (2.5$\times$ cheaper); strict 38.56 M states vs. weak counterexamples <1 s	measured; model-only for *SingleAuthority*, *SingleCommand*

TABLE IV

SERVICE-PATH MEASUREMENTS FOR RESPONDER PROPAGATION, WAN RECOVERY, COMMAND SERIALIZATION, AND RESPONDER ISOLATION.

Path / cell	Result	Interpretation
Responder-to-command propagation		
queue depth = 1 ($n = 30$)	median 710.1 ms; p95 832.1 ms	outbox-to-command journal path
queue depth = 10 ($n = 30$)	median 639.4 ms; p95 719.5 ms	outbox-to-command journal path
queue depth = 100 ($n = 30$)	median 931.5 ms; p95 1045.5 ms	outbox-to-command journal path
Command-to-core WAN recovery		
partition = 1 s ($n = 30$)	median 0.75 s; p95 0.92 s	support-path backhaul, not command authority
partition = 10 s ($n = 30$)	median 0.91 s; p95 1.04 s	support-path backhaul, not command authority
partition = 60 s ($n = 30$)	median 0.23 s; p95 1.05 s	support-path backhaul, not command authority
Command journal serialization		
direct `/accept` ($n = 29$)	median 356.3 events/s; p95 491.7	linearization point: command accept + journal write only
end-to-end via outbox ($n = 29$)	median 159.4 events/s; p95 163.9	full responder outbox $\rightarrow$ push-batch $\rightarrow$ command path
Command commit during responder isolation		
direct command accept path ($n = 20$)	20/20 commits; seq. 1–20; 0 duplicate seqs.	direct `/accept/event-batch` path; median/p95 latency 5.76/7.32 ms

TABLE V

RESPONDER PROPAGATION UNDER *tc-netem* IMPAIRMENT, 20 MEASUREMENTS PER CELL; ALL CELLS COMPLETE WITH NO DROPS.

Profile	Queue depth	Median	p95
Stressed	1	1215 ms	1494 ms
Stressed	10	2860 ms	3355 ms
Degraded	1	3647 ms	5329 ms
Degraded	10	8104 ms	10895 ms

reach the journal exactly once under retry, process restart, and impairment typical of a mesh network. Evidence: measured.

Propagation latency stays sub-second across queue depths 1–100 (Table IV).

Replaying 50 unique *client_event_id* values five times each leaves exactly 50 journal rows across $n = 30$ runs: forwarding is at-least-once and deduplicated at the journal, so submission is effectively exactly-once at the linearization point.

Across process restart, 50 outbox submissions survive a responder-Postgres power cycle before forwarding, then forward with no duplicates and consecutive *event_seq*, reaching journal consistency about 2.6 s after recovery. This is the runtime witness for R2.

Under both *tc-netem* profiles, all 80 measurements (20 per cell at queue depths 1 and 10) complete with no drops; latency degrades gracefully, with the heaviest tail at degraded qd-10 (median 8.1 s, p95 10.9 s, Table V).

D. M4: fenced promotion

M4 (fenced promotion with stale-epoch rejection) serves R3 under command transfer, exercising the C2 mechanism. Claim: a fenced promotion preserves single authority across an epoch change and rejects stale-epoch traffic before any incident-journal work. Evidence: measured on the service path; *SingleAuthority* and *SingleCommand* are model-only because the promotion harness is single-edge.

Protocol cost is single-digit milliseconds across all three phases of a transition (Figure 4), measured over $n = 300$ events (100 per phase). Commit phases A (pre-transition) and C (post-transition) have medians 6.3 ms (p95 14.0 ms) and 5.7 ms (p95 10.7 ms). Phase B, the stale-rejection path, is 2.5$\times$ cheaper (median 2.5 ms, p95 5.6 ms) by construction: the fence rejects stale traffic before the incident-journal write, so the saving is a skipped write, not a faster check on the write path. Operator confirmation sits in the critical path by design, bounded below by seconds of deliberation under stress; the protocol contribution is orders of magnitude smaller, so promotion wall-clock is an operator cost, not a network cost.

In five live service-path runs, phase A commits 20/20 events at epoch 1, phase B returns 20/20 HTTP 409 rejections for the stale epoch and adds zero incident-journal rows, and phase C commits 20/20 events at epoch 2. The incident journal holds 200 rows total, split 100/100 between epochs, with no *event_seq* value appearing under both.

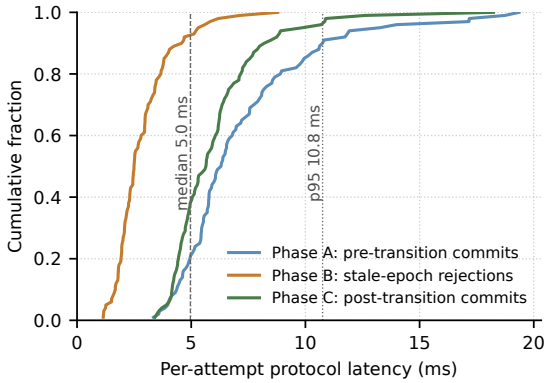

Fig. 4. Fenced-promotion protocol cost by phase. Phase A is pre-transition commits at epoch e, phase B is stale-epoch rejections at the boundary, phase C is post-transition commits at epoch $e+1$. $n = 300$ events (100 per phase). The cost shown is the protocol path only; the operator-confirm step is excluded by design.

TABLE VI
FALSIFICATION-COST ASYMMETRY. THE STRICT VARIANT VERIFIES SIX SAFETY INVARIANTS OVER TENS OF MILLIONS OF DISTINCT STATES; EACH WEAK VARIANT PRODUCES A COUNTEREXAMPLE IN HUNDREDS TO THOUSANDS OF DISTINCT STATES UNDER A SECOND OF WALL-CLOCK.

Variant	Outcome	States	Time
strict	all six invariants pass	38.56 M	13.0 min
weak	*SingleAuthority* violated	185	0.85 s
weak	*EpochAuthorityCoupling* violated	144	0.77 s
weak	*NoForkedJournal* violated	1.0 k	0.83 s

Table VI contrasts the cost of verifying the strict variant with the cost of falsifying the weak ones. The strict configuration ($V = 3$, $J = 7$, $C = 7$, $P = 3$) verifies six safety invariants over 38.56 M distinct states in 13 minutes. Each weak variant fails in under a second: counterexample traces have length 3, 3, and 5 for *SingleAuthority*, *EpochAuthorityCoupling*, and *NoForkedJournal*, found after 185, 144, and 1,015 distinct states respectively. A Hypothesis state-machine test mirrors this in Python over 2000 examples of length 50: no strict violation, and an on-demand weak counterexample. Without the fence, single-authority fails immediately.

E. Scope-delimiting paths

The remaining three experiments bound what AAL does not promise; none evaluates a mechanism.

1) Raft leadership follows the reachable quorum: This tests the R3-relevant mismatch between consensus leadership and operational command authority. The baseline is the three-voter `hashicorp/raft` journal-commit cluster in `baseline-raft/`: the HTTP leader accepts event batches, applies each event through Raft, and the in-memory FSM assigns *event_seq* values with *client_event_id* deduplication.

For each behavioral run, the harness first records the current Raft leader L_0, treats it as the node currently holding write authority, and hard-cuts it from the other two voters by disconnecting its container from the shared Docker network. The reachable majority then elects L_1 and accepts writes there (Table VII); election timing is randomized, so reported latencies are sample statistics over unseeded runs. This is correct Raft behavior: authority moves to a node that can form a quorum [38]. It is also the mechanism mismatch for R3, because Raft has no input by which an operator can pin the writer to an externally designated command edge.

The same table also reports a no-partition commit-path context measurement using the same single-event command-journal workload shape as the prototype's direct accept-path throughput run. This row is not a speed comparison. The Raft baseline uses in-memory log, snapshot, and FSM stores, while AAL commits to durable Postgres; the row is included only to bound the cost of this particular baseline path under its weaker storage contract.

2) CRDT semantics fail for authority-bearing data: This tests the C1 claim that mergeable-by-default replication cannot encode authority for non-commutative incident decisions: we attempt to preserve R3 on a CRDT-LWW baseline and observe where it fails. It is a semantic falsification, not a speed comparison. The throughput figures in § IV-B and § IV-C accept writes under different durability and convergence models, so they bound a design trade-off in different units rather than rank the implementations.

The CRDT-LWW baseline is four FastAPI replicas using Hanssen's operation-based Last-Writer-Wins Element Set with physical-clock timestamps [29]. It preserves local acceptance but loses authority-bearing conflicts: concurrent status updates converge to the latest timestamp, and a later update resurrects an element deleted by another replica. The prototype service path currently orders and fences these events rather than rejecting them semantically; that gap is a limitation in § V-B.

The infrastructure is asymmetric: the CRDT baseline is in-memory with no background anti-entropy, converging only through harness-triggered `full_sync()` after traffic stops, whereas the prototype carries a durable Postgres outbox. These asymmetries favor CRDT where it already performs better; we keep them because the test is only whether mergeable-by-default semantics can encode authority. Following the evidence taxonomy of § IV, Table VIII reports measured AAL behavior, model-level AAL safety, and measured CRDT-LWW merge semantics.

3) Backhaul recovers within about a second of WAN heal: This measures command-to-core forwarding after a WAN heal: a narrowly targeted *tc* cut of the command-to-core path only, so command Postgres and responder reachability stay available and the measurement isolates backhaul flush time (Table IV). Recovery is bounded by the forward poll loop and backlog, not outage length.

Together, these results place the authority boundary where § III does, at the cost (detailed in § V-B) of no automatic failover and lower write availability for authority-bearing operations, concentrated at operator-driven command transitions.

TABLE VII

RAFT CONSENSUS BASELINE COMPARISON. THE BEHAVIORAL PARTITION RESULT IS THE R3-RELEVANT FINDING; THE COMMIT-PATH ROW IS CONTEXT ONLY BECAUSE THE RAFT BASELINE IS IN-MEMORY WHILE AAL USES DURABLE POSTGRES.

Scenario	Measured result	Interpretation
Event sequencing sanity	leader batch accepted 1 event, deduplicated 1, last seq. 1; journal count 1	Confirms the Raft FSM assigns *event_seq* and enforces *client_event_id* idempotency.
Leader-minority partition	12/12 runs elected $L_1 \neq L_0$; 12/12 committed 5 writes at L_1; new-leader median 2.23 s, p95 3.02 s	L_0 writes while isolated: stall then apply timeout 12/12; authority follows the reachable quorum, not an external command designation.
Commit-path context	Raft in-memory leader accept median 1342.9 events/s, p95 1555.8 events/s ($n = 29$, one warmup excluded); AAL durable command accept median 356.3 events/s, p95 491.7 events/s ($n = 29$, one warmup excluded)	Different storage contracts: Raft uses in-memory log/snapshot/FSM, while AAL commits to Postgres; this is not a speed ranking.

TABLE VIII

SEMANTIC COMPARISON WITH A HANSSEN-STYLE OPERATION-BASED CRDT-LWW BASELINE. THE TABLE SEPARATES MEASURED SERVICE-PATH RESULTS FROM MODEL-LEVEL AAL GUARDS; THE RELABELLED CONVERGENCE ROW REPORTS POST-QUIESCENCE CONVERGENCE AFTER HARNESS-TRIGGERED ANTI-ENTROPY, NOT LIVE NETWORK-PARTITION IMPAIRMENT.

Scenario	AAL result	CRDT-LWW result	Interpretation
Field-edit propagation	AAL bridge median 639.4 ms at qd=10; 931.5 ms at qd=100	CRDT qd=10 stressed/degraded 814.1 ms/2.54 s; qd=100 degraded 3.97 s	Both preserve durable propagation; CRDT anti-entropy is not an authority boundary.
Concurrent status edit	Current AAL service path sequences unique events; semantic rejection is model-level/not implemented in the measured prototype	30/30 LWW runs converge; winner counts contained: 30; median 12.8 ms	LWW accepts both writes and materializes only the latest value; losing op is only recoverable from the op log.
Delete/update race	Current AAL service path has no delete/update semantic guard; limitation disclosed	30/30 runs resurrect the element; median convergence 12.7 ms	Matches Hanssen's stated LWW delete-semantics caveat.
No-partition throughput	AAL command-journal commit rate median 356.3 events/s (single writer)	CRDT aggregate local accept rate across three writers median 349.2 events/s	Different units; reports write-availability trade-off, not direct speedup.
Post-quiescence convergence (60 s outage, $n = 10$)	AAL command-to-core support-path recovery median 227.0 ms after a real *tc* WAN cut ($n = 30$)	CRDT all-replica convergence median 45.4 ms after anti-entropy resumes (no link impairment during the wait)	Comparable in outage duration, not transport mechanism; AAL keeps command-local ordering, CRDT converges once anti-entropy is exchanged.

V. SCOPE AND LIMITATIONS

A. Threat Model and Security Scope

The target deployment assumes an organizational CA or identity provider, enrolled devices and users, and backend trust in certificate-derived identity only when the reverse proxy and service boundary are correctly configured. It handles replayed field submissions through *client_event_id* idempotency and limits tablet reachability through local edge APIs and network segmentation.

Several threats remain outside the prototype evaluation. A stolen or compromised tablet can be revoked only when nodes regain contact with the authority infrastructure or receive an updated trust bundle; instantaneous revocation during total disconnection is not provided. A stolen vehicle node is more serious because it may hold cached data. If that node is the command edge, it also holds operational authority. The architecture relies on device encryption, operational fencing, and audit rather than Byzantine tolerance. Because authority is explicit, a captured or buggy command edge can poison the incident journal while it holds authority: a central trade-off of making command authority explicit.

The design also does not address a compromised organizational CA, Byzantine command behavior, physical tamper resistance beyond deployment controls, or audit-log integrity against a fully compromised node. These are deployment and hardening questions, not properties established by the emulation.

B. Evaluation Boundaries and Non-claims

The CRDT-LWW comparison is well-defined on the executable service path and is comparable in outage duration but not in transport mechanism. § IV-E2 and Table VIII explain why the absent background gossip and live link impairment favor the CRDT baseline.

For the two invariant-violation scenarios, the CRDT result is measured on its service path while the AAL guarantee rests on the TLA+ model (§ IV-D). Four of the six invariants have service-path witnesses in the live runs: *EpochAuthorityCoupling*, *NoForkedJournal*, *DurabilityAcrossPromotion*, and *LocalIdempotency*. The other two, *SingleAuthority* and *SingleCommand*, remain model-only because the promotion harness is single-edge. The service path enforces epoch fencing but does not yet implement semantic rejection for those conflicts.

The prototype evaluation runs on a single-host Docker Compose emulation with *tc-netem* impairment. This isolates protocol behavior and keeps the artifact reproducible, but it does not claim physical Rajant mesh behavior, measured WireGuard or mTLS overhead, or end-to-end Android validation. The workload is single-incident and limited-scale.

Promotion is operator-initiated. The paper does not claim automatic failover or storage-layer leader election after silent

command death; an operator runbook is part of the operational procedure. The service-path promotion check is also single-edge: it shows that stale-epoch traffic cannot poison the current command journal after an epoch change, not that a full multi-edge promotion has been measured under all operator races. End-to-end promotion wall-clock with a live operator-confirm step is out of scope for the reported emulation; only the protocol-side cost is measured (§ IV-D), and the operator-confirm floor is asserted by design rather than measured.

The strict-versus-weak model check compares fenced and unfenced promotion abstractions: relaxing epoch coupling or journal uniqueness yields short counterexamples, while the strict model checks the stated invariants. This is a bounded step toward comparing operator-initiated promotion with automated failover, not field evidence for operator handoff.

VI. Conclusion

Authority-aligned linearization rests on an operational fact that consensus and CRDT designs cannot assume: at every moment, one principal is designated as authoritative. The incident-journal linearization point is the command edge, the vehicle edge node that currently holds that authority. Reference data, field submissions, materialized state, and audit forwarding sit outside that boundary on weaker contracts. Command transfer uses fenced promotion rather than leader election.

The model checks and the prototype agree on the safety boundary. Under strict fencing, the model permits no two writers and no forked journal; the weak variant produces the expected split-brain witness, and command-edge writes proceed independently of responder reachability. The CRDT-LWW comparison shows the trade-off: CRDT replicas accept local writes at multiple nodes and converge quickly after anti-entropy exchange. AAL, by contrast, preserves one incident journal and treats semantic conflict rejection as part of the linearization boundary. The cost is no automatic failover and lower write availability for authority-bearing operations.

Future work is to deploy the prototype on a vehicle-mounted mesh, exercise fenced promotion across multiple command edges, implement service-path semantic rejection for authoritative conflicts, and add crash-boundary and Android end-to-end tests. The harder comparison is empirical, not architectural: measure the design against CoNICE-class consensus and CRDT-based emergency systems on workloads where command authority is contested rather than designated.

References

[1] M. Wiśniewski, K. Szwarc, and W. Skomra, "Continuity of essential services as an emerging challenge for societal resilience," *IEEE Access*, vol. 11, pp. 44 614–44 635, 2023.

[2] OECD, "Digital security and resilience in critical infrastructure and essential services," OECD Digital Economy Papers, Paris, 2019.

[3] B. Karaman, I. Bastürk, S. Taskin, E. Zeydan, F. Kara, E. A. Beyazıt, M. Camelo, E. Björnson, and H. Yanikomeroglu, "Solutions for sustainable and resilient communication infrastructure in disaster relief and management scenarios," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 716–760, 2026.

[4] E. G. Zimbelman, A. T. Hudak *et al.*, "Lost in the woods: Forest vegetation, and not topography, most affects the connectivity of mesh radio networks for public safety," *PLOS ONE*, 2022.

[5] Q. Wang, S. Wang *et al.*, "An overview of emergency communication networks," *Remote Sensing*, 2023.

[6] N. Andreassen, O. J. Borch, E. Ikonen *et al.*, "Information sharing and emergency response coordination," *Safety Science*, 2020.

[7] A. Marshall, C.-A. Wilson, and A. P. Dale, "Telecommunications and natural disasters in rural Australia: The role of digital capability in building disaster resilience," *Journal of Rural Studies*, vol. 100, p. 102996, 2023.

[8] M. Murphy, "Digital transformation for crisis volunteerism: A study in the aftermath of the swedish forest fires crisis in 2018," Licentiate thesis, Linköping University, 2021. [Online]. Available: https://www.diva-portal.org/smash/get/diva2:1570107/FULLTEXT01.pdf

[9] D. K. Allen, S. Karanasios, and A. Norman, "Information sharing and interoperability: The case of major incident management," *European Journal of Information Systems*, vol. 23, no. 4, pp. 418–432, 2014.

[10] Räddningstjänsten Storgöteborg, "Råd och anvisning 201: Insatsplanering vid farlig verksamhet," Operational guidance document, 2023, revised 2023-10-25; valid from 2018-12-01. [Online]. Available: https://www.rsgbg.se/globalassets/dokument/rad-och-anvisningar/rad-och-anvisning-201---insatsplanering-vid-farlig-verksamhet.pdf

[11] ——, "Kommunal Plan för räddningsinsätser – Del I: Allmän del," Municipal rescue-services plan, 2025, dated 2019-01-02; revised 2025-06-09. [Online]. Available: https://www.rsgbg.se/globalassets/dokument/foretag-och-organisationer/seveso/rsg-kommunal-plan-for-raddningsinsats-del-i---allman-del.pdf

[12] S. Pilemalm, D. Andersson, and K. Yousefi Mojir, "Enabling organizational learning from rescue operations: The Swedish rescue services incident reporting system," *International Journal of Emergency Services*, vol. 3, no. 2, pp. 101–117, 2014.

[13] A. Westman *et al.*, "Mobilisation of emergency services for chemical incidents in Sweden – a multi-agency focus group study," *Scandinavian Journal of Trauma, Resuscitation and Emergency Medicine*, vol. 29, p. 99, 2021.

[14] D. P. Moynihan, "Combining structural forms in the search for policy tools: Incident command systems in U.S. crisis management," *Governance*, vol. 21, no. 2, pp. 205–229, 2008.

[15] ——, "The network governance of crisis response: Case studies of incident command systems," *Journal of Public Administration Research and Theory*, vol. 19, no. 4, pp. 895–915, 2009.

[16] B. Nowell, T. Steelman, A.-L. K. Velez, and Z. Yang, "The structure of effective governance of disaster response networks: Insights from the field," *The American Review of Public Administration*, vol. 48, no. 7, pp. 699–715, 2018.

[17] B. Nowell and T. Steelman, "Beyond ICS: How should we govern complex disasters in the United States?" *Journal of Homeland Security and Emergency Management*, vol. 16, no. 2, p. 20180067, 2019.

[18] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.

[19] P. Helland, "Life beyond distributed transactions: An apostate's opinion," *Communications of the ACM*, vol. 60, no. 2, pp. 46–54, 2017.

[20] L. Lamport, "The part-time parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.

[21] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference*, 2014, pp. 305–320.

[22] M. Balakrishnan, D. Malkhi, T. Wobber *et al.*, "Tango: Distributed data structures over a shared log," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 325–340.

[23] R. van Renesse and F. B. Schneider, "Chain replication for supporting high throughput and availability," in *6th Symposium on Operating Systems Design and Implementation (OSDI 04)*. San Francisco, CA: USENIX Association, Dec. 2004, pp. 91–104. [Online]. Available: https://www.usenix.org/conference/osdi-04/chain-replication-supporting-high-throughput-and-availability

[24] P. S. Almeida, A. Shoker, and C. Baquero, "Efficient state-based CRDTs by delta-mutation," in *International Conference on Networked Systems (NETYS)*, 2014.

[25] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, "Verifying strong eventual consistency in distributed systems," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–28, 2017.

[26] J. Haas, V. Balegas *et al.*, "Lore: A programming model for verifiably safe local-first software," *ACM Transactions on Programming Languages and Systems*, 2023.
[27] M. Köhler, G. Salvaneschi *et al.*, "Consistent local-first software: Enforcing safety and invariants for local-first applications," *IEEE Transactions on Software Engineering*, 2025.
[28] N. V. Lewchenko, G. Kaki, and B.-Y. E. Chang, "Bolt-on strong consistency: Specification, implementation, and verification," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA, pp. 1604–1631, 2025.
[29] Ø. Hanssen, "Data replication for distributed emergency management systems," in *Norsk IKT-konferanse for forskning og utdanning (NIK)*, 2025.
[30] P. S. Almeida, "Approaches to conflict-free replicated data types," *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, 2024.
[31] Y. Mao, G. Zhang, Z. Liu, P. Nasirifard, S. Tijanic, and H.-A. Jacobsen, "Making CRDTs not so eventual," *Proceedings of the VLDB Endowment*, vol. 18, no. 2, pp. 349–362, 2024.
[32] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019.
[33] P. K. Paul *et al.*, "CRIMP: Here crisis mapping goes offline," *Journal of Network and Computer Applications*, 2019.
[34] M. Jahanian and K. K. Ramakrishnan, "CoNICE: Consensus in intermittently-connected environments by exploiting naming with application to emergency response," *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1926–1939, 2022.
[35] A. Kumbhar, F. Koohifar, I. Guvenc, and B. Mueller, "A survey on legacy and emerging technologies for public safety communications," *IEEE Communications Surveys & Tutorials*, 2015.
[36] M. F. H. Sagor, A. Haroon, R. Stoleru, S. Bhunia, A. Altaweel, M. Chao, L. Jin, M. Maurice, and R. Blalock, "DistressNet-NG: A resilient data storage and sharing framework for mobile edge computing in cyber-physical systems," *ACM Transactions on Cyber-Physical Systems*, vol. 8, no. 3, pp. 37:1–37:31, 2024.
[37] C. Li, D. Porto, A. Clement, J. Gehrke, N. Preguiça, and R. Rodrigues, "Making geo-replicated systems fast as possible, consistent when necessary," in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. Hollywood, CA: USENIX Association, 2012, pp. 265–278. [Online]. Available: https://www.usenix.org/conference/osdi12/technical-sessions/presentation/li
[38] Y. Li, Y. Fan, L. Zhang, and J. Crowcroft, "RAFT consensus reliability in wireless networks: Probabilistic analysis," *IEEE Internet of Things Journal*, vol. 10, no. 14, pp. 12 839–12 853, 2023.